

2.5 Second Order Methods

- The previous notebook only looks at algorithms requiring the value of the function and 1st derivative of the function
- In this notebook we will look at algorithms of finding minimum of a function using value, derivative as well as Hessian (double derivative) information.
- These methods are sometimes called the second order methods

Load plotting library and define structure to store intermediate data

In [2]: `using CairoMakie`

In [3]: `struct Point1D
 x::Float64
 fvalue::Float64
end`

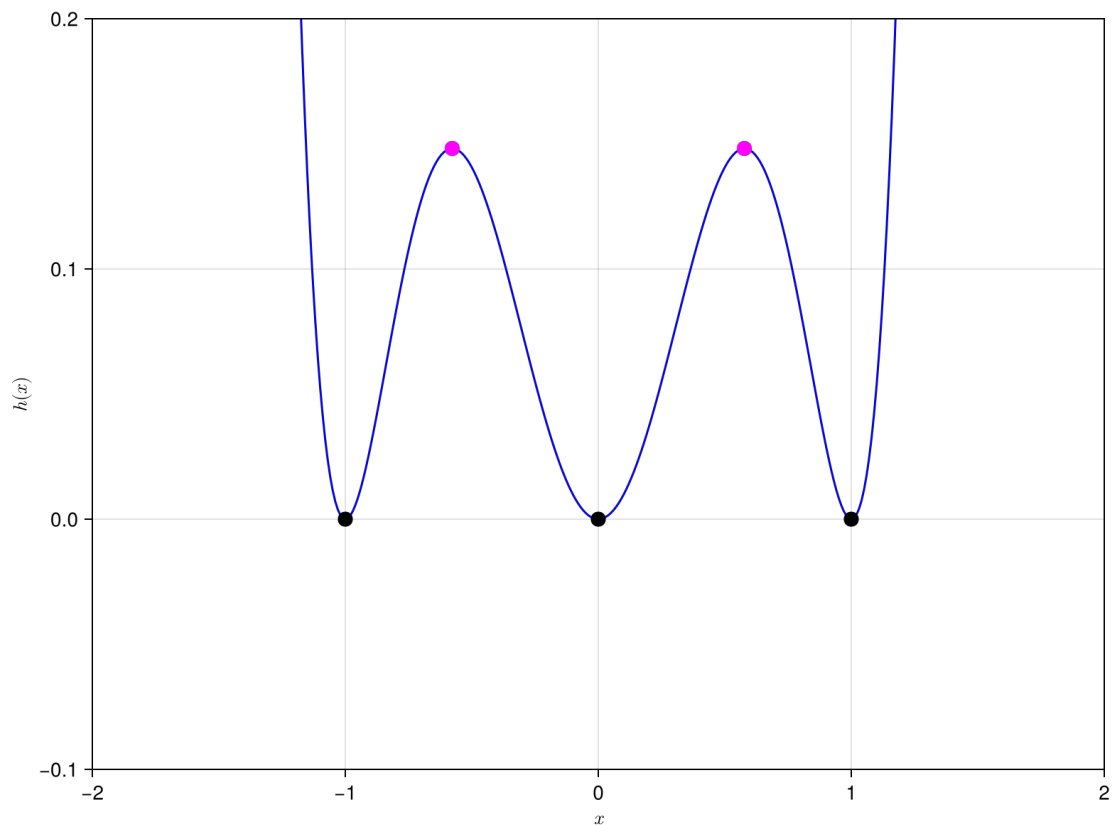
Example 2.5.1

Let's try to find the minimum of the function $h(x) = (x^3 - x)^2$. Make it a habit to always plot the graph (if you can) and visualise how the function looks like

```
In [5]: h(x)=(x^3-x)^2
        ∇h(x)=2*(x^3-x)*(3x^2-1);
        Hessiah(x)=12*x*(x^3-x)+2*(3*x^2-1)^2; # Second derivative
        xplot=-2:0.01:2
        yplot=[h(x) for x in xplot];
        fig = Figure(backgroundcolor = :transparent, size=(800,600))
        ax = Axis(fig[1, 1], xlabel = L"x", ylabel = L"$h(x)$", limits=(-2,2,-0.1,
        lines!(ax,xplot,yplot;color=:blue)
        scatter!(ax,[0, -1, 1],h.([0, -1, 1]);color=:black,markersize=15)
        scatter!(ax,[1/sqrt(3), -1/sqrt(3)],h.([1/sqrt(3), -1/sqrt(3)]);color=:ma

        # Save the figure
        save("../figures/ManyMinimums01.png", fig)

        display(fig)
```



CairoMakie.Screen{IMAGE}

- The local minimum of the function is located at $x = -1, 0, 1$ indicated by the black dots.
- Note that for this function there is a maximum at $x = \pm\sqrt{1/3}$ indicated by the magenta dots.

We have used the standard gradient descent method to find the minimum in the previous notebook. Now we look at the Newton method. Consider Taylor series of a function $f(x)$ expanded about $x = x_0$

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2.$$

Differentiate with respect to x gives

$$\frac{df}{dx}(x) = f'(x_0) + f''(x_0)(x - x_0)$$

At a local minimum, $df/dx = 0$.

$$x = x_0 - f'(x_0)/f''(x_0)$$

So Newton's method can be thought of as fitting a 2nd order Taylor's polynomial about point x_0 and then find the minimum of this 2nd order Taylor's polynomial. The update equation for the Newton method is then

$$x^{(k+1)} = x^{(k)} - g^{(k)}/r^{(k)}$$

where $g^{(k)} = \frac{df}{dx}(x^{(k)})$ and $r^{(k)} = \frac{d^2f}{dx^2}(x^{(k)})$ is the first and second derivative of the function $f(x)$ evaluated at the previous guess $x^{(k)}$.

I will now graphically illustrate the Newton method.

```
In [8]: fig = Figure(backgroundcolor = :transparent, size=(800,600))
ax = Axis(fig[1, 1], xlabel = "x", ylabel = L"$h(x)$", limits=(-2,2,-0.1,0
lines!(ax,xplot,yplot;color=:blue,label="True function")

x0=0.9
f2ndOrderTaylor(x,x0)=h(x0)+∇h(x0)*(x-x0)+(Hessianh(x0)/2.0)*(x-x0)^2
lines!(ax,xplot,f2ndOrderTaylor.(xplot,x0);color=:green, label="Taylors S

xnewton=x0-∇h(x0)/Hessianh(x0)

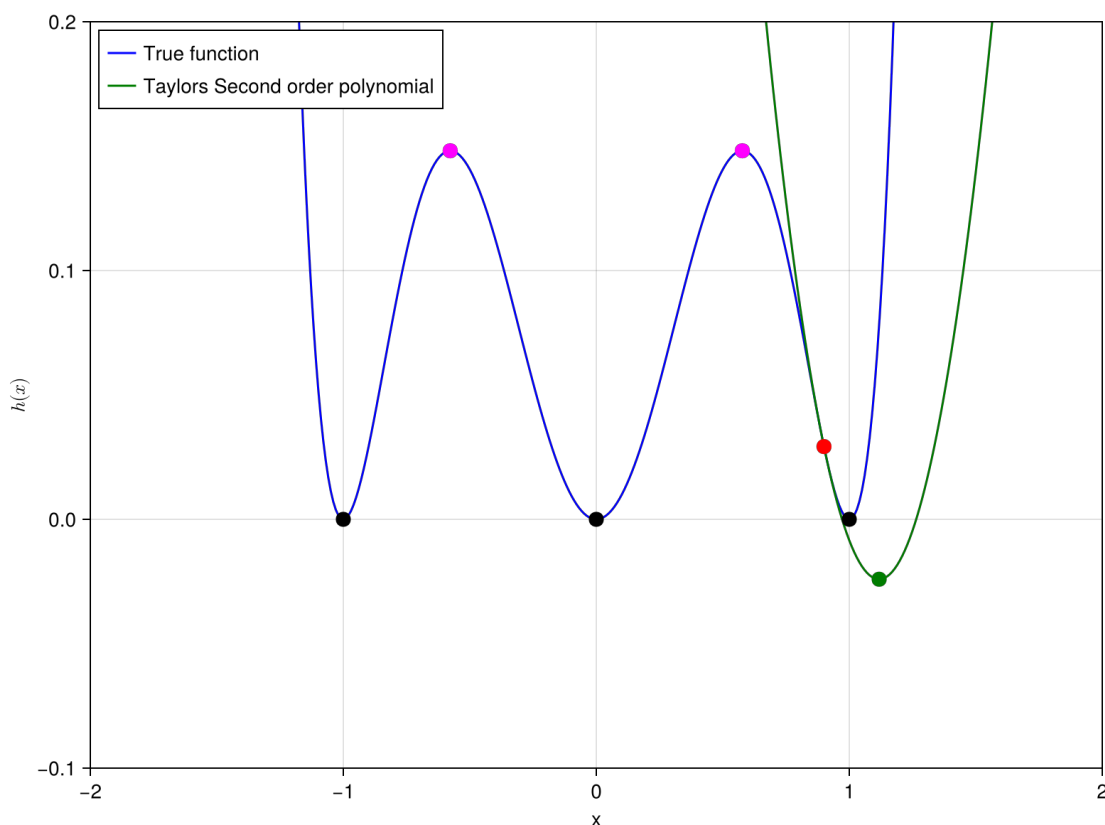
scatter!(ax,xnewton,f2ndOrderTaylor(xnewton,x0);color=:green,markersize=1
scatter!(ax,x0,h(x0);color=:red,markersize=15)

scatter!(ax,[0, -1, 1],h.([0, -1, 1]);color=:black,markersize=15)
scatter!(ax,[1/sqrt(3), -1/sqrt(3)],h.([1/sqrt(3), -1/sqrt(3)]);color=:ma

axislegend(ax,position=:lt)

# Save the figure
save("../figures/ManyMinimums02.png", fig)

display(fig)
```



CairoMakie.Screen{IMAGE}

- For this example, the true function is blue line above. The true location of the minimum is given by the black dot.
- If the initial guess is at $x_0 = 0.9$ (given by the red dot in the figure above), then, after the first iteration, the Newton method will predict the location to be at the green dot (which is the fitted 2nd order Taylor polynomial about $x_0 = 0.9$)
- The Newton method assumes that the green dot is closer to the black dot than the original red dot.

We can code up the update equations for the Newton method

$$x^{(k+1)} = x^{(k)} - g^{(k)} / r^{(k)}$$

$(g^{(k)} = \frac{df}{dx}(x^{(k)})$ and $r^{(k)} = \frac{d^2f}{dx^2}(x^{(k)})$ is the first and second derivative of the function $f(x)$ evaluated at the previous guess $x^{(k)}$) below

```
In [9]: x0=0.9 #initial guess of x0
x=x0
data_Newton=[Point1D(x0,h(x0))]
for i in 1 : 20
    x-=Hessianh(x)\∇h(x) #update equation for the Newton method
    push!(data_Newton,Point1D(x,h(x)))
end
```

```
In [10]: data_Newton
```

```
21-element Vector{Point1D}:
 Point1D(0.9, 0.029240999999999975)
 Point1D(1.1180383415069102, 0.07813169165795499)
 Point1D(1.0365899460770065, 0.00596684442404486)
 Point1D(1.0049270862755146, 9.85477253796149e-5)
 Point1D(1.000106051277926, 4.500180878731018e-8)
 Point1D(1.000000050578152, 1.023259945612181e-14)
 Point1D(1.00000000000000115, 5.33269971929404e-28)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
 Point1D(1.0, 0.0)
```

We will now plot the convergence to see how fast the Newton method converges

```
In [13]: fig_conv = Figure()

ax_conv1 = Axis(fig_conv[1, 1], xlabel = L"x", ylabel = L"f(x)", limits=(-
lines!(ax_conv1,xplot,h.(xplot))
scatter!(ax_conv1,getproperty.(data_Newton,:x),getproperty.(data_Newton,:
color=1:length(data_Newton),
```

```

    strokewidth=1,
    strokecolor=:black,
    colormap = :Blues,
    markersize=15)

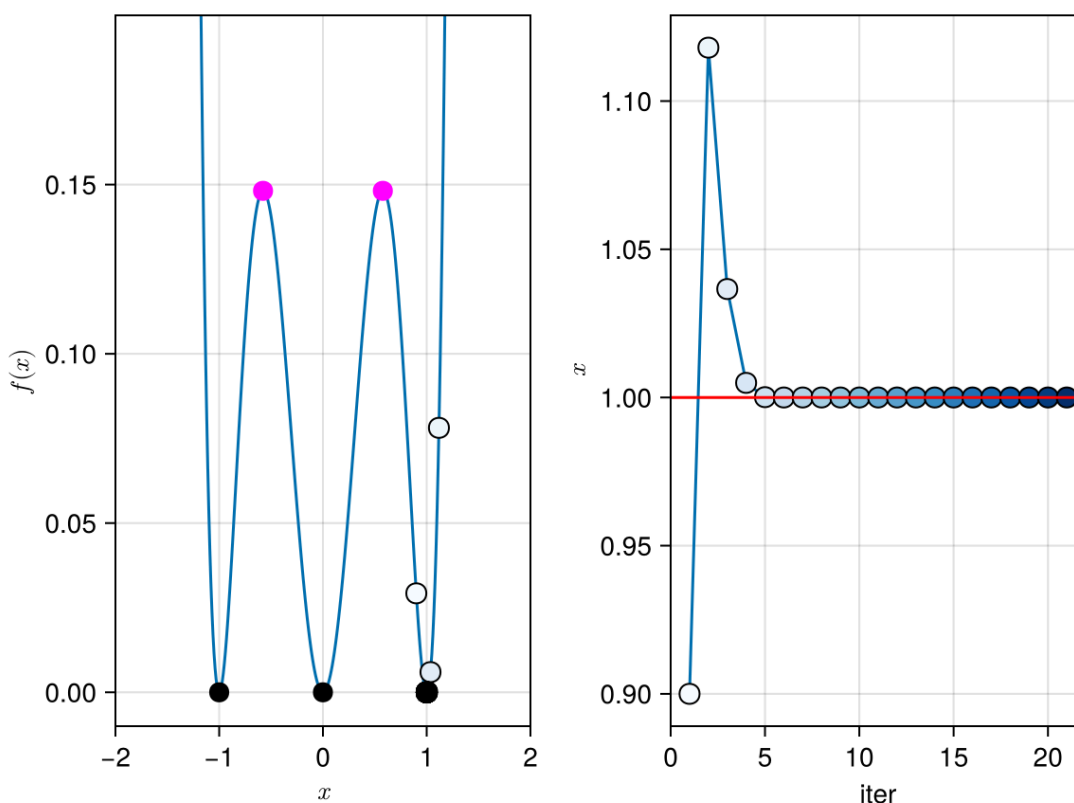
ax_conv2 = Axis(fig_conv[1, 2], xlabel = "iter", ylabel = L"x")
lines!(ax_conv2, getproperty.(data_Newton, :x), label="Gradient Descent")
scatter!(ax_conv2, getproperty.(data_Newton, :x);
    color=1:length(data_Newton),
    strokewidth=1,
    strokecolor=:black,
    colormap = :Blues,
    markersize=15)
hlines!(ax_conv2, 1.0; color=:red)

scatter!(ax_conv1, [0, -1, 1], h.([0, -1, 1]); color=:black, markersize=15)
scatter!(ax_conv1, [1/sqrt(3), -1/sqrt(3)], h.([1/sqrt(3), -1/sqrt(3)]); col
fig_conv

# Save the figure
save("../figures/ManyMinimums02.png", fig_conv)

display(fig_conv)

```



CairoMakie.Screen{IMAGE}

- For Newton method, this problem converges in about 3 iterations.

Let's try to compute the same solution using the Nesterov Momentum method and compare the rate of convergence with Newton method to see how fast it converges.

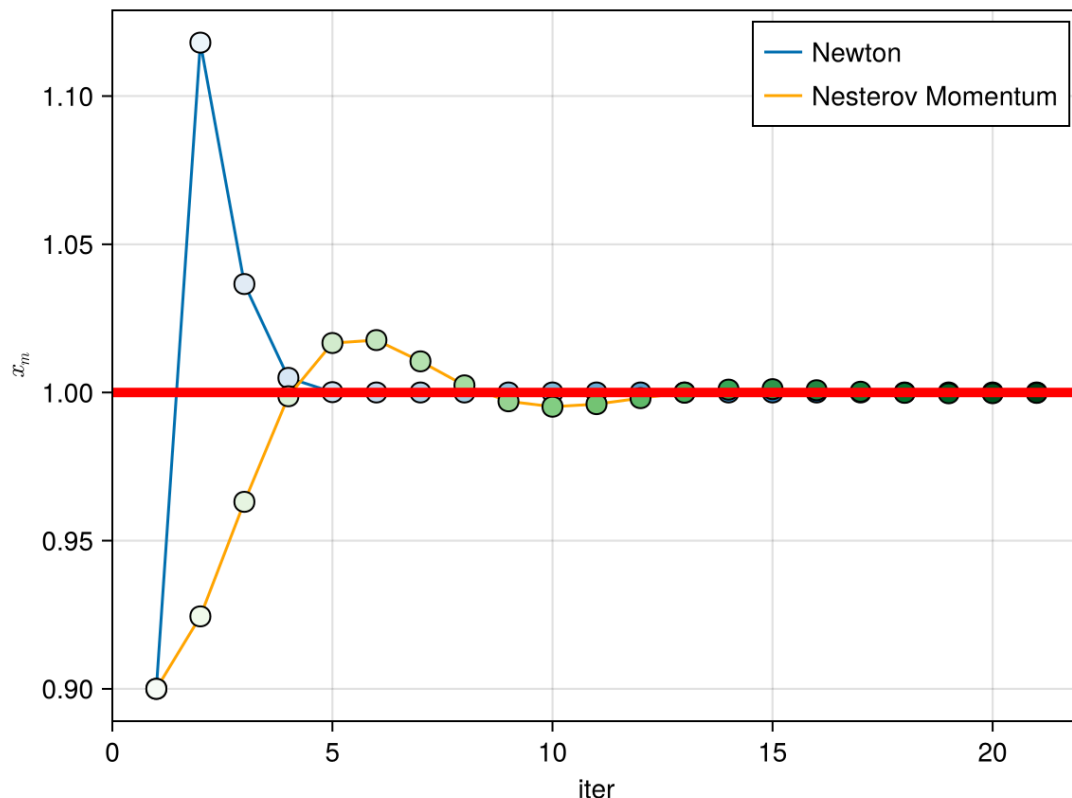
```
In [14]: x0=0.9 #initial guess of xm
alpha=0.05 #learning rate
beta=0.9
v=0.0
x=x0
data_nm=[Point1D(x0,h(x0))]
for i in 1 : 20
    v =beta*v-alpha*∇h(x+beta*v) #update equations for the Nesterov Momen
    x+=v #update equations for the Nesterov Momentum
    push!(data_nm,Point1D(x,h(x)))
end
```

```
In [15]: fig_conv = Figure()
ax_conv = Axis(fig_conv[1, 1], xlabel = "iter", ylabel = L"x_m")

lines!(ax_conv,getproperty.(data_Newton, :x),label="Newton")
scatter!(ax_conv,getproperty.(data_Newton,:x);
    color=1:length(data_Newton),
    strokewidth=1,
    strokecolor=:black,
    colormap = :Blues,
    markersize=15)

lines!(ax_conv,getproperty.(data_nm, :x),label="Nesterov Momentum",color=:red)
scatter!(ax_conv,getproperty.(data_nm,:x);
    color=1:length(data_nm),
    strokewidth=1,
    strokecolor=:black,
    colormap = :Greens,
    markersize=15)

axislegend(ax_conv, position = :rt,orientation = :vertical)
hlines!(ax_conv,1.0;color=:red,linewidth=5.0)
fig_conv
```



- The graph above shows that Nesterov momentum method take about twice as long to converge compared to Newton method.
- In general Newton method is very fast, however, there can be some pitfalls.

Beware some pitfalls of the Newton Method

- Note that Newton method looks for regions where the derivative (gradient) is zero
- The gradient can be zero either at minimum or at maximum.
- If we start close to the maximum Newton method will find the maximum
- Let's see what happens if we try Newton method with a different guess

```
In [16]: x0=0.4 #initial guess of x

data_Newton=[Point1D(x0,h(x0))]
for i in 1 : 20
    x0=-Hessianh(x0)\∇h(x0)
    push!(data_Newton,Point1D(x0,h(x0)))
end
```

Let's plot the convergence of Newton method

```
In [17]: fig_conv = Figure()

ax_conv1 = Axis(fig_conv[1, 1], xlabel = L"x", ylabel = L"f(x)", limits=(-
lines!(ax_conv1,xplot,h.(xplot))
scatter!(ax_conv1,getproperty.(data_Newton,:x),getproperty.(data_Newton,:
color=1:length(data_Newton),
```

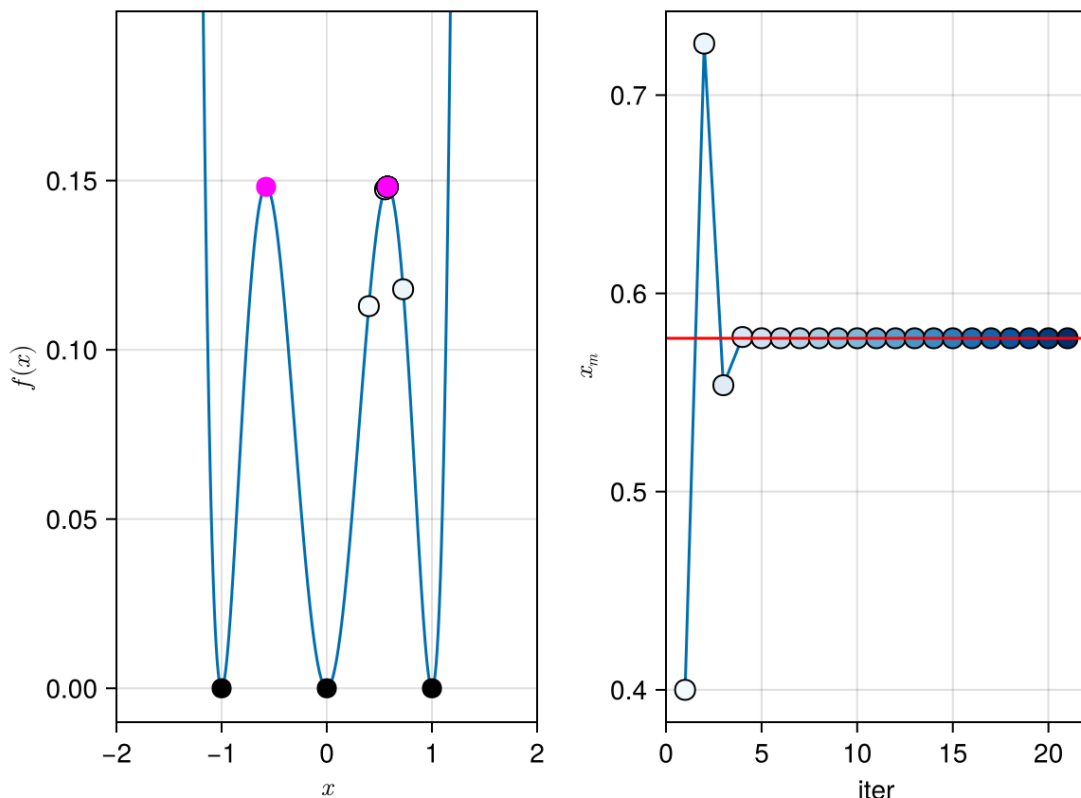
```

    linewidth=1,
    strokecolor=:black,
    colormap = :Blues,
    markersize=15)

ax_conv2 = Axis(fig_conv[1, 2], xlabel = "iter", ylabel = L"x_m")
lines!(ax_conv2, getproperty.(data_Newton, :x), label="Gradient Descent")
scatter!(ax_conv2, getproperty.(data_Newton, :x),
    color=1:length(data_Newton),
    linewidth=1,
    strokecolor=:black,
    colormap = :Blues,
    markersize=15)
hlines!(ax_conv2, +sqrt(1/3); color=:red)

scatter!(ax_conv1, [0, -1, 1], h.([0, -1, 1]); color=:black, markersize=15)
scatter!(ax_conv1, [1/sqrt(3), -1/sqrt(3)], h.([1/sqrt(3), -1/sqrt(3)]); col
fig_conv

```



If we start the calculation with initial guess $x_0 = 0.4$, the Newton method goes towards the local maximum (the magenta dot)!

We will now try Nesterov momentum method with initial guess $x_0 = 0.4$

```

In [18]: x0=0.4 #initial guess of x0
alpha=0.05 #learning rate
beta=0.9
v=0.0
x=x0
data_nm=[Point1D(x0,h(x0))]
for i in 1 : 20
    v =beta*v-alpha*∇h(x+beta*v) #update equations for the Nesterov Momen

```



```

x+=v #update equations for the Nesterov Momentum
push!(data_nm,Point1D(x,h(x)))
end

```

```

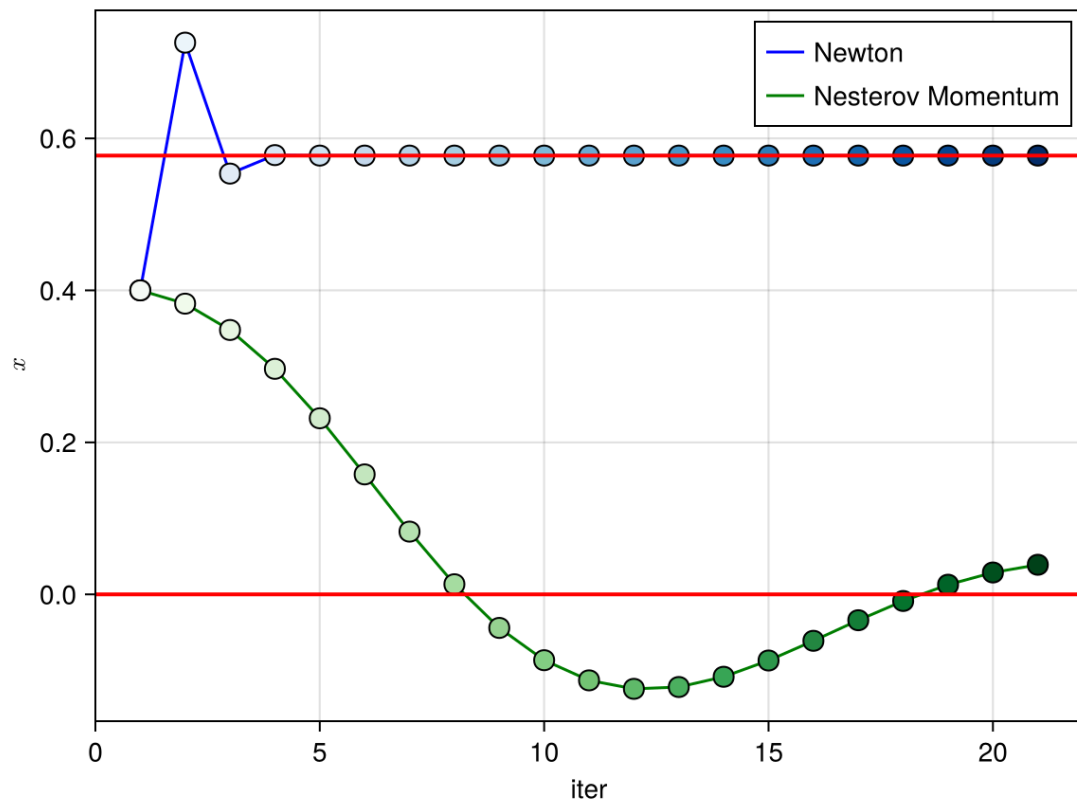
In [19]: fig_conv = Figure()
ax_conv = Axis(fig_conv[1, 1], xlabel = "iter", ylabel = L"x")

lines!(ax_conv,getproperty.(data_Newton, :x),color=:blue,label="Newton")
scatter!(ax_conv,getproperty.(data_Newton, :x);
        color=1:length(data_Newton),
        strokewidth=1,
        strokecolor=:black,
        colormap = :Blues,
        markersize=15)

lines!(ax_conv,getproperty.(data_nm, :x),color=:green,label="Nesterov Mom")
scatter!(ax_conv,getproperty.(data_nm, :x);
        color=1:length(data_nm),
        strokewidth=1,
        strokecolor=:black,
        colormap = :Greens,
        markersize=15)

axislegend(ax_conv, position = :rt,orientation = :vertical)
hlines!(ax_conv,sqrt(1.0/3.0);color=:red,linewidth=2.0)
hlines!(ax_conv,0.0;color=:red,linewidth=2.0)
fig_conv

```



- The Nesterov Momentum method converges to the correct minimum at $x = 0.0$
- The Newton method, even though faster, converges to the local maximum at $x = 1/\sqrt{3}$

- In general, it is usually found that first order method (such as Nesterov momentum) can usually always converge to a minimum but but the convergence can take a long time.
- If you are close to the minimum, second order methods (such as Newton method) converges very fast. However, if you are far away from the minimum, second order methods can converge to the wrong answer.
- In practice, it is advisable to try first order methods initially to try to get closer to the minimum. Once you care close to the minimum, you would switch to the second order method.